```
#!/usr/bin/env python3
"""
Parlay Builder (7+ legs, expandable sports list)
------------------------------------------------
Pulls live odds from The Odds API (https://the-odds-api.com) for every sport on
YOUR list (stored in sports.json next to this script), pools all the candidate
legs together, and builds the best parlay of 7 or more picks.

Method:
  1. Remove the vig from each book's market -> fair probabilities.
  2. Average across books -> consensus win probability.
  3. Find the best price per outcome (line shopping), edge = prob*odds - 1.
  4. Take the best leg from each sport on your list first (so the parlay really
     spans your list), then fill the rest with the highest-probability legs
     overall. Max one leg per game to avoid correlated picks.

Setup:
  export ODDS_API_KEY=your_key        pip install requests

Manage your sports list:
  python parlay_builder.py --list-sports
  python parlay_builder.py --discover tennis          # find valid sport keys
  python parlay_builder.py --add-sport atp tennis_atp_us_open
  python parlay_builder.py --remove-sport ligue1
  python parlay_builder.py --disable-sport mls        # keep it but skip it
  python parlay_builder.py --enable-sport mls

Build a parlay:
  python parlay_builder.py                            # 7 legs, next 24h
  python parlay_builder.py --legs 10 --hours 36
  python parlay_builder.py --sports nba nfl mlb       # one-off subset
  python parlay_builder.py --per-sport-min 2          # >=2 legs per sport if pos
  python parlay_builder.py --per-sport-min 0          # purely best legs overall
  python parlay_builder.py --watch 300                # refresh every 5 min
  python parlay_builder.py --demo                     # offline test
"""
import argparse
import json
import os
import random
import sys
import time
from collections import defaultdict
```

```
from datetime import datetime, timedelta, timezone

try:
    import requests
except ImportError:
    requests = None

BASE = "https://api.the-odds-api.com/v4/sports"
CONFIG = os.path.join(os.path.dirname(os.path.abspath(__file__)), "sports.json")
MIN_LEGS = 7

DEFAULT_SPORTS = {
    "nfl": "americanfootball_nfl",
    "ncaaf": "americanfootball_ncaaf",
    "nba": "basketball_nba",
    "mlb": "baseball_mlb",
    "nhl": "icehockey_nhl",
    "mls": "soccer_usa_mls",
    "epl": "soccer_epl",
    "laliga": "soccer_spain_la_liga",
    "seriea": "soccer_italy_serie_a",
    "bundesliga": "soccer_germany_bundesliga",
    "ligue1": "soccer_france_ligue_one",
    "ucl": "soccer_uefa_champs_league",
}


# ---------------------------------------------------------- sports list I/O
def load_sports():
    if not os.path.exists(CONFIG):
        data = {k: {"key": v, "enabled": True} for k, v in DEFAULT_SPORTS.items()
        save_sports(data)
        return data
    with open(CONFIG) as f:
        return json.load(f)


def save_sports(data):
    with open(CONFIG, "w") as f:
        json.dump(data, f, indent=2)


def need_requests():
    if requests is None:
        sys.exit("pip install requests")
```

```
def api_key():
    k = os.environ.get("ODDS_API_KEY")
    if not k:
        sys.exit("Set ODDS_API_KEY (free key at https://the-odds-api.com).")
    return k


def manage_sports(args):
    """Handle list/add/remove/enable/disable/discover. Returns True if handled."""
    sports = load_sports()
    handled = False
    if args.add_sport:
        name, key = args.add_sport[0].lower(), args.add_sport[1]
        sports[name] = {"key": key, "enabled": True}
        print(f"Added '{name}' -> {key}")
        handled = True
    for n in args.remove_sport or []:
        print(("Removed " if sports.pop(n.lower(), None) else "Not found: ") + n)
        handled = True
    for n in args.disable_sport or []:
        if n.lower() in sports:
            sports[n.lower()]["enabled"] = False
        handled = True
    for n in args.enable_sport or []:
        if n.lower() in sports:
            sports[n.lower()]["enabled"] = True
        handled = True
    if handled:
        save_sports(sports)
    if args.discover is not None:
        need_requests()
        r = requests.get(BASE, params={"apiKey": api_key()}, timeout=20)
        for s in r.json():
            text = f'{s["key"]} {s["title"]} {s.get("group","")}'.lower()
            if args.discover.lower() in text:
                print(f'{s["key"]:<40} {s["title"]}  {"(in season)" if s["active"
        handled = True
    if args.list_sports or handled:
        print("\nYour sports list:")
        for n, v in sports.items():
            print(f'  [{"x" if v["enabled"] else " "}] {n:<12} {v["key"]}')
        handled = True
    return handled
```

```
# ---------------------------------------------------------------- data fetch
def fetch_events(sport_key, key, regions, markets):
    r = requests.get(
        f"{BASE}/{sport_key}/odds/",
        params={"apiKey": key, "regions": regions, "markets": markets,
                "oddsFormat": "decimal", "dateFormat": "iso"},
        timeout=20,
    )
    if r.status_code != 200:
        print(f"  [warn] {sport_key}: HTTP {r.status_code} {r.text[:100]}", file=
        return []
    left = r.headers.get("x-requests-remaining")
    if left:
        print(f"  [{sport_key}] API requests remaining: {left}", file=sys.stderr)
    return r.json()


def demo_events(sport_names):
    random.seed(7)
    now = datetime.now(timezone.utc)
    events = []
    for i in range(40):
        sport = sport_names[i % len(sport_names)]
        p = random.uniform(0.35, 0.82)
        books = []
        for b in ["BookA", "BookB", "BookC", "BookD"]:
            q = min(0.95, max(0.05, p + random.uniform(-0.03, 0.03)))
            books.append({"title": b, "markets": [{"key": "h2h", "outcomes": [
                {"name": f"{sport}-H{i}", "price": round(1 / (q * 1.05), 3)},
                {"name": f"{sport}-A{i}", "price": round(1 / ((1 - q) * 1.05), 3)
        events.append({"id": f"d{i}", "home_team": f"{sport}-H{i}", "away_team":
                       "commence_time": (now + timedelta(hours=random.randint(1,
                       .strftime("%Y-%m-%dT%H:%M:%SZ"),
                       "bookmakers": books, "_sport": sport})
    return events


# ---------------------------------------------------------------- analysis
def build_legs(events, max_hours, min_books):
    now = datetime.now(timezone.utc)
    cutoff = now + timedelta(hours=max_hours)
    legs = []
    for ev in events:
        start = datetime.fromisoformat(ev["commence_time"].replace("Z", "+00:00")
        if not (now <= start <= cutoff):
            continue
```

```
        agg = defaultdict(lambda: {"probs": [], "best": (0.0, "")})
        for bk in ev.get("bookmakers", []):
            for mk in bk.get("markets", []):
                outs = mk.get("outcomes", [])
                if len(outs) < 2 or any(o["price"] <= 1 for o in outs):
                    continue
                inv = [1.0 / o["price"] for o in outs]
                total = sum(inv)
                for o, imp in zip(outs, inv):
                    a = agg[(mk["key"], o["name"], o.get("point"))]
                    a["probs"].append(imp / total)
                    if o["price"] > a["best"][0]:
                        a["best"] = (o["price"], bk["title"])
        for (market, name, point), a in agg.items():
            if len(a["probs"]) < min_books:
                continue
            prob = sum(a["probs"]) / len(a["probs"])
            odds, book = a["best"]
            legs.append({
                "event_id": ev["id"], "game": f'{ev["away_team"]} @ {ev["home_tea
                "sport": ev.get("_sport", ""), "start": start, "market": market,
                "pick": name + (f" {point:+g}" if market == "spreads" else
                                f" {point:g}" if market == "totals" else ""),
                "prob": prob, "odds": odds, "book": book,
                "edge": prob * odds - 1, "n_books": len(a["probs"]),
            })
    return legs


def pick_parlay(legs, n_legs, min_prob, allow_neg_ev, per_sport_min):
    pool = [l for l in legs if l["prob"] >= min_prob]
    if not allow_neg_ev:
        pool = [l for l in pool if l["edge"] >= -0.005]
    pool.sort(key=lambda l: (l["prob"], l["edge"]), reverse=True)

    chosen, used = [], set()

    def take(l):
        chosen.append(l)
        used.add(l["event_id"])

    # Pass 1: guarantee representation from every sport on the list
    by_sport = defaultdict(list)
    for l in pool:
        by_sport[l["sport"]].append(l)
    for _ in range(per_sport_min):
```

```
        for sport, ls in by_sport.items():
            if len(chosen) >= n_legs:
                break
            for l in ls:
                if l["event_id"] not in used:
                    take(l)
                    break
    # Pass 2: fill the remainder with the best legs overall
    for l in pool:
        if len(chosen) >= n_legs:
            break
        if l["event_id"] not in used:
            take(l)

    chosen.sort(key=lambda l: l["prob"], reverse=True)
    alts = [l for l in pool if l["event_id"] not in used][:6]
    return chosen, alts


def american(dec):
    return f"+{round((dec - 1) * 100)}" if dec >= 2 else f"-{round(100 / (dec - 1


def report(parlay, alts, n_legs, sports_seen):
    print("\n" + "=" * 78)
    print(f" BEST {len(parlay)}-LEG PARLAY   ({datetime.now():%Y-%m-%d %H:%M})")
    print("=" * 78)
    if len(parlay) < n_legs:
        print(f" Only {len(parlay)} qualifying legs found. Try --hours, --min-pro
        print(" --allow-negative-ev, or add more sports.")
    p = o = 1.0
    for i, l in enumerate(parlay, 1):
        p *= l["prob"]
        o *= l["odds"]
        print(f"{i:>2}. [{l['sport'].upper():<10}] {l['game']}")
        print(f"    {l['market'].upper():<7} {l['pick']:<28} win%={l['prob']*100:
              f"best={american(l['odds'])} @ {l['book']}  edge={l['edge']*100:+.1
    if parlay:
        counts = defaultdict(int)
        for l in parlay:
            counts[l["sport"].upper()] += 1
        print("-" * 78)
        print(" Sports mix               :", ", ".join(f"{k} x{v}" for k, v in co
        print(f" Combined win probability : {p*100:.2f}%  (about 1 in {1/p:.0f})"
        print(f" Parlay payout (best book): {american(o)}  ({o:.2f}x)")
        print(f" Expected value per $1    : {p*o-1:+.3f}")
```

```
        print(" NOTE: real parlays use ONE book's odds, so actual payout will be
    if alts:
        print("\n Alternates:")
        for l in alts:
            print(f"   - [{l['sport'].upper()}] {l['game']} | {l['pick']} | "
                  f"{l['prob']*100:.1f}% @ {american(l['odds'])}")
    print("\n No model guarantees a win; each added leg multiplies the book's mar
    print(" Bet only what you can afford to lose. Help: 1-800-GAMBLER.\n")


# ---------------------------------------------------------------------- main
def run(args, sports):
    names = args.sports or [n for n, v in sports.items() if v["enabled"]]
    events = []
    if args.demo:
        events = demo_events(names)
    else:
        need_requests()
        key = api_key()
        for n in names:
            sk = sports.get(n, {}).get("key", n)  # allow raw keys too
            print(f"Fetching {n} ({sk}) ...", file=sys.stderr)
            for ev in fetch_events(sk, key, args.regions, args.markets):
                ev["_sport"] = n
                events.append(ev)
    legs = build_legs(events, args.hours, args.min_books)
    print(f"Analyzed {len(events)} games across {len(names)} sports, "
          f"{len(legs)} candidate legs.", file=sys.stderr)
    parlay, alts = pick_parlay(legs, args.legs, args.min_prob,
                               args.allow_negative_ev, args.per_sport_min)
    report(parlay, alts, args.legs, names)


def main():
    ap = argparse.ArgumentParser(description="Build the best 7+ leg parlay across
    ap.add_argument("--legs", type=int, default=MIN_LEGS, help="number of legs (m
    ap.add_argument("--hours", type=float, default=24, help="only games starting
    ap.add_argument("--sports", nargs="+", help="one-off subset of names from you
    ap.add_argument("--per-sport-min", type=int, default=1,
                    help="guarantee at least N legs per sport when available (0 =
    ap.add_argument("--min-prob", type=float, default=0.55, help="min win prob pe
    ap.add_argument("--min-books", type=int, default=3, help="min books quoting a
    ap.add_argument("--regions", default="us")
    ap.add_argument("--markets", default="h2h,spreads,totals")
    ap.add_argument("--allow-negative-ev", action="store_true")
    ap.add_argument("--watch", type=int, default=0, help="refresh every N seconds
```

```
    ap.add_argument("--demo", action="store_true", help="offline fake data")
    # sports list management
    ap.add_argument("--list-sports", action="store_true")
    ap.add_argument("--add-sport", nargs=2, metavar=("NAME", "ODDS_API_KEY"))
    ap.add_argument("--remove-sport", nargs="+", metavar="NAME")
    ap.add_argument("--enable-sport", nargs="+", metavar="NAME")
    ap.add_argument("--disable-sport", nargs="+", metavar="NAME")
    ap.add_argument("--discover", nargs="?", const="", metavar="TEXT",
                    help="search the API for available sport keys")
    args = ap.parse_args()

    if manage_sports(args):
        return
    if args.legs < MIN_LEGS:
        sys.exit(f"--legs must be {MIN_LEGS} or more.")
    sports = load_sports()
    while True:
        run(args, sports)
        if not args.watch:
            break
        time.sleep(args.watch)


if __name__ == "__main__":
    main()
```